



Deep Learning

Tutorial 15: Long Short-Term Memory (LSTM)

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # **000000494171**

Semester: **4th**

Department of Robotics & Artificial Intelligence

School of Mechanical & Manufacturing Engineering

NUST

Tasks:

Compare it with Simple RNN
Make a model of LSTM for sentiment analysis.

Implementing the Task in Pytorch:

```
# =====  
  
# Tutorial 15 – Long Short-Term Memory (LSTM)  
  
# Implemented in PyTorch | Ready for Google Colab  
  
# =====  
  
# Objectives:  
  
# • Understand the architecture of LSTM  
  
# • How to implement the LSTM in PyTorch  
  
# • Visualize the results  
  
#  
  
# TASKS COVERED:  
  
# ✓ Google Stock Price Prediction with LSTM  
  
# ✓ Comparison with Simple RNN on the same task  
  
# ✓ LSTM model for Sentiment Analysis  
  
# =====  
  
# — 0. Install dependencies —————  
  
# Run this cell first in Google Colab:  
  
# !pip install yfinance --quiet
```

```
import torch

import torch.nn as nn

import torch.optim as optim

from torch.utils.data import Dataset, DataLoader, TensorDataset

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler


try:

    import yfinance as yf

    YF_AVAILABLE = True

except ImportError:

    YF_AVAILABLE = False

    print("yfinance not installed. Run: !pip install yfinance --quiet")


torch.manual_seed(42)

np.random.seed(42)


device = "cuda" if torch.cuda.is_available() else "cpu"

print(f"Using device: {device}")
```

```

# =====

# SECTION 1 – GOOGLE STOCK PRICE PREDICTION WITH LSTM

# =====

print("\n" + "="*60)

print("SECTION 1 – LSTM: Google Stock Price Prediction")

print("="*60)


# — 1.1 Fetch data —————

if YF_AVAILABLE:

    df = yf.download('GOOGL', start='2010-01-01', end='2024-12-01', auto_adjust=True)

    print(df.head())

    close_prices = df['Close'].values.reshape(-1, 1)

else:

    # Fallback: synthetic sine-based price data so the rest of the code runs

    print("Using synthetic data as fallback.")

    t = np.linspace(0, 40 * np.pi, 3500)

    close_prices = (50 * np.sin(t) + 100 + np.random.randn(3500) * 5).reshape(-1, 1)


# — 1.2 Normalise —————

scaler = MinMaxScaler(feature_range=(0, 1))

scaled_data = scaler.fit_transform(close_prices)


# Train / test split 80 / 20

```

```
train_size = int(len(scaled_data) * 0.8)
```

```
train_data = scaled_data[:train_size]
```

```
test_data = scaled_data[train_size:]
```

```
# — 1.3 Create sequences —————
```

```
def create_sequences(data, seq_length):
```

```
    X, y = [], []
```

```
    for i in range(len(data) - seq_length):
```

```
        X.append(data[i : i + seq_length])
```

```
        y.append(data[i + seq_length])
```

```
    return np.array(X), np.array(y)
```

```
SEQUENCE_LENGTH = 60 # Use past 60 days to predict next day
```

```
X_train, y_train = create_sequences(train_data, SEQUENCE_LENGTH)
```

```
X_test, y_test = create_sequences(test_data, SEQUENCE_LENGTH)
```

```
print(f"X_train shape: {X_train.shape}, y_train shape: {y_train.shape}")
```

```
# Convert to tensors
```

```
X_train_t = torch.tensor(X_train, dtype=torch.float32).to(device)
```

```
y_train_t = torch.tensor(y_train, dtype=torch.float32).to(device)
```

```
X_test_t = torch.tensor(X_test, dtype=torch.float32).to(device)
```

```
y_test_t = torch.tensor(y_test, dtype=torch.float32).to(device)
```

```
# — 1.4 LSTM Model —————
```

```
class LSTMPredictor(nn.Module):
```

```
    def __init__(self, input_size=1, hidden_size=50, num_layers=2, output_size=1):
```

```
        super().__init__()
```

```
        self.lstm = nn.LSTM(input_size, hidden_size,
```

```
                             num_layers=num_layers,
```

```
                             batch_first=True,
```

```
                             dropout=0.2)
```

```
        self.fc1 = nn.Linear(hidden_size, 25)
```

```
        self.relu = nn.ReLU()
```

```
        self.fc2 = nn.Linear(25, output_size)
```

```
    def forward(self, x):
```

```
        out, _ = self.lstm(x)    # (B, T, H)
```

```
        out = out[:, -1, :]    # last time step
```

```
        out = self.relu(self.fc1(out))
```

```
        out = self.fc2(out)
```

```
        return out
```

```
# — 1.5 Simple RNN Model (for comparison) —————
```

```
class SimpleRNNPredictor(nn.Module):
```

```

def __init__(self, input_size=1, hidden_size=50, num_layers=2, output_size=1):

    super().__init__()

    self.rnn = nn.RNN(input_size, hidden_size,

                       num_layers=num_layers,

                       batch_first=True,

                       nonlinearity='tanh')

    self.fc1 = nn.Linear(hidden_size, 25)

    self.relu = nn.ReLU()

    self.fc2 = nn.Linear(25, output_size)


def forward(self, x):

    out, _ = self.rnn(x)

    out = out[:, -1, :]

    out = self.relu(self.fc1(out))

    out = self.fc2(out)

    return out

```

— 1.6 Training function —————

```

def train_stock_model(model, X_train, y_train, X_test, y_test,

                      epochs=10, batch_size=32, lr=1e-3, model_name="Model"):

    criterion = nn.MSELoss()

    optimizer = optim.Adam(model.parameters(), lr=lr)

```

```

dataset = TensorDataset(X_train, y_train)

loader = DataLoader(dataset, batch_size=batch_size, shuffle=False)

train_losses, val_losses = [], []

for epoch in range(epochs):

    model.train()

    epoch_loss = 0

    for xb, yb in loader:

        optimizer.zero_grad()

        pred = model(xb)

        loss = criterion(pred, yb)

        loss.backward()

        optimizer.step()

        epoch_loss += loss.item() * len(xb)

    epoch_loss /= len(X_train)

    model.eval()

    with torch.no_grad():

        val_pred = model(X_test)

        val_loss = criterion(val_pred, y_test).item()

    train_losses.append(epoch_loss)

```



```

    val_losses.append(val_loss)

    if (epoch + 1) % max(1, epochs // 5) == 0:
        print(f" [{model_name}] Epoch {epoch+1}/{epochs} "
              f"Train Loss: {epoch_loss:.6f} Val Loss: {val_loss:.6f}")

    return train_losses, val_losses

# — 1.7 Train both models —————

EPOCHS = 10

lstm_model = LSTMPredictor().to(device)
rnn_model = SimpleRNNPredictor().to(device)

print("\nTraining LSTM ...")

lstm_train_loss, lstm_val_loss = train_stock_model(
    lstm_model, X_train_t, y_train_t, X_test_t, y_test_t,
    epochs=EPOCHS, batch_size=32, lr=1e-3, model_name="LSTM")

print("\nTraining Simple RNN ...")

rnn_train_loss, rnn_val_loss = train_stock_model(
    rnn_model, X_train_t, y_train_t, X_test_t, y_test_t,
    epochs=EPOCHS, batch_size=32, lr=1e-3, model_name="SimpleRNN")

```

```

# — 1.8 Evaluate & visualise predictions —————

def get_predictions(model, X_test):

    model.eval()

    with torch.no_grad():

        preds = model(X_test).cpu().numpy()

    return scaler.inverse_transform(preds)

lstm_preds = get_predictions(lstm_model, X_test_t)

rnn_preds = get_predictions(rnn_model, X_test_t)

y_test_actual = scaler.inverse_transform(y_test_t.cpu().numpy())

# Plot 1 – Actual vs LSTM vs RNN predictions

plt.figure(figsize=(14, 5))

plt.plot(y_test_actual, label="Actual Price", color="blue", linewidth=1.5)

plt.plot(lstm_preds, label="LSTM Predicted", color="red", linewidth=1.2, linestyle="--")

plt.plot(rnn_preds, label="SimpleRNN Predicted", color="green", linewidth=1.2, linestyle=":")

plt.title("GOOGL Stock Price Prediction – LSTM vs Simple RNN")

plt.xlabel("Days"); plt.ylabel("Price (USD)")

plt.legend(); plt.tight_layout()

plt.savefig("Tutorial15_stock_predictions.png", dpi=150)

plt.show()

print("Saved: Tutorial15_stock_predictions.png")

```

```
# Plot 2 – Training loss comparison
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
```

```
axes[0].plot(lstm_train_loss, label="LSTM train", color="red")
```

```
axes[0].plot(rnn_train_loss, label="SimpleRNN train", color="green")
```

```
axes[0].set_title("Training Loss"); axes[0].set_xlabel("Epoch")
```

```
axes[0].set_ylabel("MSE Loss"); axes[0].legend()
```

```
axes[1].plot(lstm_val_loss, label="LSTM val", color="red", linestyle="--")
```

```
axes[1].plot(rnn_val_loss, label="SimpleRNN val", color="green", linestyle="--")
```

```
axes[1].set_title("Validation Loss"); axes[1].set_xlabel("Epoch")
```

```
axes[1].set_ylabel("MSE Loss"); axes[1].legend()
```

```
plt.suptitle("Tutorial 15 – LSTM vs Simple RNN Loss Comparison")
```

```
plt.tight_layout(); plt.savefig("Tutorial15_loss_comparison.png", dpi=150)
```

```
plt.show()
```

```
print("Saved: Tutorial15_loss_comparison.png")
```

```
# — 1.9 Predict next-day price —————
```

```
last_sequence = scaled_data[-SEQUENCE_LENGTH:].reshape(1, SEQUENCE_LENGTH, 1)
```

```
last_seq_t = torch.tensor(last_sequence, dtype=torch.float32).to(device)
```

```
lstm_model.eval()
```

```
with torch.no_grad():
```

```

next_day_scaled = lstm_model(last_seq_t).cpu().numpy()

next_day_price = scaler.inverse_transform(next_day_scaled)

print(f'\nPredicted next-day price for GOOGL (LSTM): ${next_day_price[0][0]:.2f}')

```

— 1.10 MSE comparison —

```

lstm_mse = np.mean((lstm_preds - y_test_actual) ** 2)

rnn_mse = np.mean((rnn_preds - y_test_actual) ** 2)

print(f'\nTest MSE – LSTM: {lstm_mse:.4f} | SimpleRNN: {rnn_mse:.4f}')

```

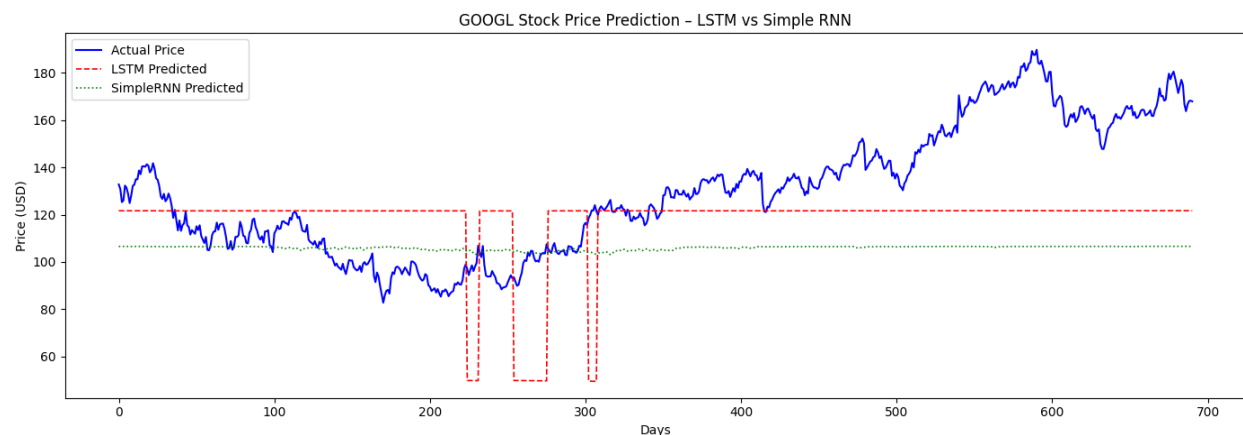
```

=====
SECTION 1 - LSTM: Google Stock Price Prediction
=====
[*****100%*****] 1 of 1 completed
Price      Close      High      Low      Open      Volume
Ticker     GOOGL      GOOGL      GOOGL      GOOGL      GOOGL
Date
2010-01-04  15.555864  15.624368  15.493567  15.560828  78169752
2010-01-05  15.487362  15.582919  15.426554  15.566538  120067812
2010-01-06  15.096946  15.533776  15.049788  15.533776  158988852
2010-01-07  14.745496  15.140133  14.709508  15.125240  256315428
2010-01-08  14.942069  14.972597  14.621643  14.693373  188783028
X_train shape: (2942, 60, 1), y_train shape: (2942, 1)

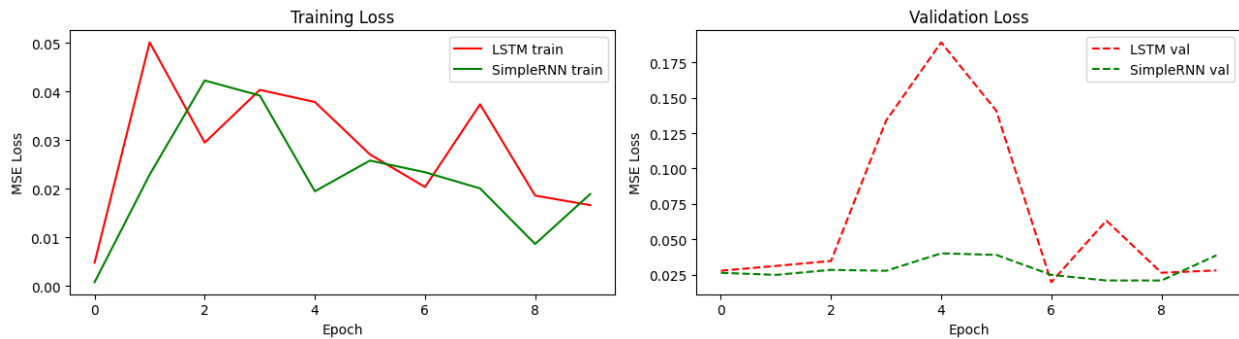
Training LSTM _
[LSTM] Epoch 2/10 Train Loss: 0.050078 Val Loss: 0.031291
[LSTM] Epoch 4/10 Train Loss: 0.040300 Val Loss: 0.133778
[LSTM] Epoch 6/10 Train Loss: 0.027029 Val Loss: 0.140821
[LSTM] Epoch 8/10 Train Loss: 0.037329 Val Loss: 0.063255
[LSTM] Epoch 10/10 Train Loss: 0.016596 Val Loss: 0.028113

Training Simple RNN _
[SimpleRNN] Epoch 2/10 Train Loss: 0.022870 Val Loss: 0.024989
[SimpleRNN] Epoch 4/10 Train Loss: 0.039138 Val Loss: 0.027862
[SimpleRNN] Epoch 6/10 Train Loss: 0.025761 Val Loss: 0.039087
[SimpleRNN] Epoch 8/10 Train Loss: 0.020038 Val Loss: 0.021033
[SimpleRNN] Epoch 10/10 Train Loss: 0.018874 Val Loss: 0.038821

```



Tutorial 15 – LSTM vs Simple RNN Loss Comparison



```
Predicted next-day price for GOOGL (LSTM): $121.68
```

```
Test MSE - LSTM: 900.8296 | SimpleRNN: 1243.9485
```

```
# =====
```

```
# SECTION 2 – TASK: LSTM FOR SENTIMENT ANALYSIS
```

```
# =====
```

```
print("\n" + "="*60)
```

```
print("SECTION 2 – LSTM: Sentiment Analysis (Task Solution)")
```

```
print("="*60)
```

```
# — 2.1 Custom sentiment dataset —————
```

```
sentiment_sentences = [
```

```
    "I love this product it is amazing",
```

```
    "This is the best movie I have ever seen",
```

```
    "Absolutely wonderful experience highly recommend",
```

```
    "The food was delicious and the service was great",
```

```
    "I am so happy with this purchase",
```

```
    "Great quality and fast delivery",
```

```
    "This is terrible I hate it",
```

```

"Worst experience ever do not buy",
"The product broke after one day very disappointed",
"Poor quality and bad customer service",
"I will never buy from this company again",
"Complete waste of money awful product",
"The movie was okay nothing special",
"Average product does the job",
"It was fine not great not terrible",
]
sentiment_labels = [1,1,1,1,1,1, 0,0,0,0,0,0, 0,0,1] # 1=positive, 0=negative

```

— 2.2 Tokenise —————

```
VOCAB_SIZE_SA = 500
```

```
PAD_IDX = 0
```

```
# Simple word-level tokeniser
```

```
word_counts = {}
```

```
for sent in sentiment_sentences:
```

```
    for w in sent.lower().split():
```

```
        word_counts[w] = word_counts.get(w, 0) + 1
```

```
sorted_words = sorted(word_counts, key=word_counts.get, reverse=True)
```

```
sa_word2idx = {"<PAD>": 0, "<UNK>": 1}
```

```
for w in sorted_words[:VOCAB_SIZE_SA - 2]:
```

```
    sa_word2idx[w] = len(sa_word2idx)
```

```
MAX_SA_LEN = 12
```

```
def encode_sa(sent):
```

```
    tokens = sent.lower().split()
```

```
    idxs = [sa_word2idx.get(t, 1) for t in tokens]
```

```
    # pad / truncate
```

```
    idxs = idxs[:MAX_SA_LEN] + [0] * max(0, MAX_SA_LEN - len(idxs))
```

```
    return idxs
```

```
X_sa = torch.tensor([encode_sa(s) for s in sentiment_sentences], dtype=torch.long).to(device)
```

```
y_sa = torch.tensor(sentiment_labels, dtype=torch.float32).to(device)
```

```
# — 2.3 LSTM Sentiment Model —————
```

```
class LSTMSentiment(nn.Module):
```

```
    def __init__(self, vocab_size, embed_dim=32, hidden_size=64, num_layers=2):
```

```
        super().__init__()
```

```
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
```

```
        self.lstm = nn.LSTM(embed_dim, hidden_size,
```

```
                               num_layers=num_layers,
```

```
                               batch_first=True,
```

```

        dropout=0.3)

    self.fc      = nn.Linear(hidden_size, 1)

    self.sigmoid = nn.Sigmoid()

def forward(self, x):

    emb      = self.embedding(x)      # (B, T, E)

    out, _    = self.lstm(emb)         # (B, T, H)

    last_out  = out[:, -1, :]         # last time step

    logit     = self.fc(last_out)      # (B, 1)

    return self.sigmoid(logit).squeeze(1) # (B,)

sa_model = LSTMSentiment(vocab_size=len(sa_word2idx)).to(device)

optimizer = optim.Adam(sa_model.parameters(), lr=0.005)

criterion = nn.BCELoss()

sa_losses = []

EPOCHS_SA = 80

for epoch in range(EPOCHS_SA):

    sa_model.train()

    optimizer.zero_grad()

    preds = sa_model(X_sa)

    loss = criterion(preds, y_sa)

```



```

loss.backward()

optimizer.step()

sa_losses.append(loss.item())

if (epoch + 1) % 20 == 0:

    print(f' [Sentiment LSTM] Epoch {epoch+1}/{EPOCHS_SA} Loss: {loss.item():.4f}')

# — 2.4 Evaluate sentiment predictions —————

sa_model.eval()

with torch.no_grad():

    probs = sa_model(X_sa).cpu().numpy()

    pred_labels = (probs >= 0.5).astype(int)

print("\n— Sentiment Analysis Results —")

for sent, true_l, pred_l, prob in zip(sentiment_sentences,

                                     sentiment_labels, pred_labels, probs):

    mark = "✓" if true_l == pred_l else "✗"

    print(f' [{mark}] '{sent[:45]:45s}' → "

          f'True: {'POS' if true_l else 'NEG'} | "

          f'Pred: {'POS' if pred_l else 'NEG'} ({prob:.2f})')

accuracy = (pred_labels == np.array(sentiment_labels)).mean()

print(f'\nTraining Accuracy: {accuracy*100:.1f}%')

```

```

# — 2.5 Test on new sentences —————

def predict_sentiment(model, sentences):

    model.eval()

    X = torch.tensor([encode_sa(s) for s in sentences],

                      dtype=torch.long).to(device)

    with torch.no_grad():

        probs = model(X).cpu().numpy()

    return probs

new_reviews = [

    "This is an incredible product I love it",

    "Terrible quality broke immediately",

    "It is decent nothing special",

]

new_probs = predict_sentiment(sa_model, new_reviews)

print("\n— New Sentence Predictions —")

for sent, p in zip(new_reviews, new_probs):

    label = "POSITIVE" if p >= 0.5 else "NEGATIVE"

    print(f' '{sent}' → {label} (score: {p:.3f})')

# — 2.6 Visualise sentiment loss —————

fig, axes = plt.subplots(1, 2, figsize=(13, 4))

axes[0].plot(sa_losses, color="#9B59B6")

```

```

axes[0].set_title("LSTM Sentiment – Training Loss")

axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("BCE Loss")


# Confidence bar chart

colors = ["#2ECC71" if l == 1 else "#E74C3C" for l in sentiment_labels]

axes[1].barh(range(len(sentiment_sentences)), probs, color=colors, edgecolor="black")

axes[1].axvline(0.5, color="black", linestyle="--", linewidth=1)

axes[1].set_yticks(range(len(sentiment_sentences)))

axes[1].set_yticklabels([s[:35] for s in sentiment_sentences], fontsize=7)

axes[1].set_xlabel("Positive Probability")

axes[1].set_title("Sentiment Prediction Confidence")


plt.suptitle("Tutorial 15 – LSTM Sentiment Analysis")

plt.tight_layout()

plt.savefig("Tutorial15_sentiment.png", dpi=150, bbox_inches="tight")

plt.show()

print("Saved: Tutorial15_sentiment.png")

print("\n✅ Tutorial 15 Complete!")

```

SECTION 2 - LSTM: Sentiment Analysis (Task Solution)

```
[Sentiment LSTM] Epoch 20/80 Loss: 0.0253
[Sentiment LSTM] Epoch 40/80 Loss: 0.0008
[Sentiment LSTM] Epoch 60/80 Loss: 0.0004
[Sentiment LSTM] Epoch 80/80 Loss: 0.0003
```

— Sentiment Analysis Results —

[✓] 'I love this product it is amazing	' → True: POS Pred: POS (1.00)
[✓] 'This is the best movie I have ever seen	' → True: POS Pred: POS (1.00)
[✓] 'Absolutely wonderful experience highly recomm	' → True: POS Pred: POS (1.00)
[✓] 'The food was delicious and the service was gr	' → True: POS Pred: POS (1.00)
[✓] 'I am so happy with this purchase	' → True: POS Pred: POS (1.00)
[✓] 'Great quality and fast delivery	' → True: POS Pred: POS (1.00)
[✓] 'This is terrible I hate it	' → True: NEG Pred: NEG (0.00)
[✓] 'Worst experience ever do not buy	' → True: NEG Pred: NEG (0.00)
[✓] 'The product broke after one day very disappoi	' → True: NEG Pred: NEG (0.00)
[✓] 'Poor quality and bad customer service	' → True: NEG Pred: NEG (0.00)
[✓] 'I will never buy from this company again	' → True: NEG Pred: NEG (0.00)
[✓] 'Complete waste of money awful product	' → True: NEG Pred: NEG (0.00)
[✓] 'The movie was okay nothing special	' → True: NEG Pred: NEG (0.00)
[✓] 'Average product does the job	' → True: NEG Pred: NEG (0.00)
[✓] 'It was fine not great not terrible	' → True: POS Pred: POS (1.00)

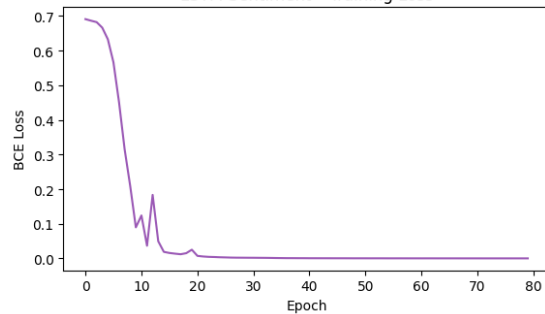
Training Accuracy: 100.0%

— New Sentence Predictions —

```
'This is an incredible product I love it' → POSITIVE (score: 0.992)
'Terrible quality broke immediately' → NEGATIVE (score: 0.000)
'It is decent nothing special' → NEGATIVE (score: 0.000)
```

Tutorial 15 - LSTM Sentiment Analysis

LSTM Sentiment - Training Loss



Sentiment Prediction Confidence

